Subprogram

- A subprogram is a **named PL/SQL block** that **performs a specific task**.
- There are two major types: Function and Procedure
    - **Function: A function must return a value.**

      FUNCTION calculate_bonus
      RETURN NUMBER
    - **Procedure: A procedure does not have to return a value.**

      PROCEDURE display_employee
    - However, procedures can return values through OUT or IN OUT parameters.

Functions

- A function is a named PL/SQL block that **performs an operation and returns exactly one value**.
- If you want to perform some **intermediate operation** and want to **return some value**, in that case use **functions**.
- While **defining the function**, you need to **specify which kind of value you are going to return**, like procedure you **can pass parameter** as well.
- Function can be **invoked by select and where clause**.
- Syntax:

```
CREATE OR REPLACE FUNCTION function_name
(  parameter_name datatype )
RETURN datatype
IS
   -- declarations
BEGIN
   -- executable statements
   RETURN value;
END;
/
```

- Example: Find the square of the number

```
CREATE OR REPLACE FUNCTION square_num
( p_num NUMBER )
RETURN NUMBER
IS
BEGIN
   RETURN p_num * p_num;
END;
/
```

    - **Explanation:**

        - CREATE OR REPLACE: Creates a new function or replaces an existing function.
        - FUNCTION square_num: Function Name
        - p_num NUMBER: The parameter receives the value supplied by the caller.
        - RETURN NUMBER: This specifies the datatype of the value returned by the function.
        - RETURN p_num * p_num;: A function must execute a RETURN statement that returns a compatible value.
    - **Execute**
        - **SELECT square_num(5) AS square FROM dual;**
    - **Output: 25**
- DUAL is a special Oracle table used when we need to execute an expression without retrieving data from a real table.

Function with Variables

- Create a function that accepts basic salary and returns salary + 10% bonus.

```
CREATE OR REPLACE FUNCTION salary_with_bonus( p_salary NUMBER)
RETURN NUMBER
IS
   v_bonus NUMBER;
   v_total NUMBER;
BEGIN
   v_bonus := p_salary * 0.10;
   v_total := p_salary + v_bonus;
   RETURN v_total;
END;
/
```

- Execute:

```
SELECT salary_with_bonus(50000) FROM dual;
```

- Output:

```
55000
```

- Select first_name, last_name, salary, salary_with_bonus (salary) from employee;

Function with IF-ELSE

- Create a function that returns the grade based on marks.

```
CREATE OR REPLACE FUNCTION get_grade(p_marks NUMBER)
RETURN VARCHAR2 IS
BEGIN
   IF p_marks >= 90 THEN
```

```
    RETURN 'A';
  ELSIF p_marks >= 80 THEN
    RETURN 'B';
  ELSIF p_marks >= 70 THEN
    RETURN 'C';
  ELSIF p_marks >= 60 THEN
    RETURN 'D';
  ELSE
    RETURN 'F';
  END IF;
END;
/
```

- Execute: SELECT get_grade(85) FROM dual;
- Output: B

Function Using a Table

- Imagine employee table is already created and some rows are there
- Function to Retrieve Employee Salary

```
CREATE OR REPLACE FUNCTION get_salary(p_emp_id NUMBER)
RETURN NUMBER
IS
  v_salary NUMBER;
BEGIN
  SELECT salary INTO v_salary FROM employee WHERE emp_id = p_emp_id;
  RETURN v_salary;
END;
/
```

- **Execute**: SELECT get_salary(102) FROM dual;
- **Output**: 60000

Function with Exception Handling

- A robust function should handle possible errors.

```
CREATE OR REPLACE FUNCTION get_salary( p_emp_id NUMBER)
RETURN NUMBER
IS
  v_salary NUMBER;
BEGIN
  SELECT salary INTO v_salary FROM employee WHERE emp_id = p_emp_id;
  RETURN v_salary;
```

```
EXCEPTION
   WHEN NO_DATA_FOUND THEN
      RETURN NULL;
   WHEN TOO_MANY_ROWS THEN
      RETURN NULL;
END;
/
```

Function with Multiple Parameters

- Calculate simple interest SI=PNR/100

```
CREATE OR REPLACE FUNCTION simple_interest
(    p_principal NUMBER,
   p_rate NUMBER,
   p_time NUMBER)
RETURN NUMBER
IS
BEGIN
   RETURN (p_principal * p_rate * p_time) / 100;
END;
/
```

- You will get: Function created.
- Test it: SELECT simple_interest(10000, 5, 2) FROM dual;
- Output: 1000

Assignment

1. Create a function that accepts an employee ID and returns the employee's salary, then calculate the annual salary
2. Create a function: department_average(department_id)
   The function should return the average salary of that department.
   Handle:
   a. Invalid department
   b. No employees
   c. NULL values appropriately

Procedures

- If you **want to reuse the code** written inside the PL/SQL block, then you have to call that portion. For that you need **some name**.
- **Name the anonymous PL/SQL** block into named block **as a database object**, so that we can reuse it whenever required.
- Procedure is a **named block** of statement. Value can be supplied through parameters. It may or may not return a value.
- A procedure is a named PL/SQL block designed to perform a particular task. Unlike a function, **a procedure does not require a RETURN value**.
- **Formal parameter is passed when we define procedure while actual parameter is passed when we invoke it.**
- **Syntax:**

```
CREATE OR REPLACE PROCEDURE procedure_name
(
   parameter_name datatype
)
IS
BEGIN
   -- statements
END;
/
```

- Create a procedure to display a welcome message.

```
CREATE OR REPLACE PROCEDURE welcome_message
IS
BEGIN
   DBMS_OUTPUT.PUT_LINE('Welcome to PL/SQL');
END;
/
```

  1. **Execute**: EXEC welcome_message;
  2. **Output**: Welcome to PL/SQL

- Alternatively:

```
BEGIN
   welcome_message;
END;
/
```

- **Different modes of passing parameter**
  1. IN parameter (default)
  2. OUT parameter
  3. IN OUT parameter

- **Example 1:**

```
Create or replace procedure test_procedure IS
Begin
dbms_output.put_line('TEST Procedure');
End;
```

Procedure created.

```
Execute test_procedure;
```

- **Example 2:**

```
CREATE OR REPLACE PROCEDURE display_square
(
   p_num IN NUMBER
)
IS
BEGIN
   DBMS_OUTPUT.PUT_LINE('Square = ' || p_num * p_num);
END;
/
```

  - **Execute:** EXEC display_square(7);
  - **Output:** Square = 49
- **IN means:** The caller sends a value into the procedure/function.
- **OUT is used when the procedure needs to send a value back to the caller.**
- **Example 3:**

```
CREATE OR REPLACE PROCEDURE calculate_square
(
   p_num IN NUMBER,
   p_result OUT NUMBER
)
IS
BEGIN
   p_result := p_num * p_num;
END;
/
```

- **Now we need a variable to receive the result.**

```
DECLARE
   v_result NUMBER;
BEGIN
   calculate_square(8, v_result);
   DBMS_OUTPUT.PUT_LINE('Square = ' || v_result);
END;
/
```

- IN OUT allows a parameter to be:
  - Passed into the procedure
  - Modified
  - Returned to the caller
- Example 4:

```
CREATE OR REPLACE PROCEDURE increase_salary
(
   p_salary IN OUT NUMBER
)
IS
BEGIN
   p_salary := p_salary + 5000;
END;
/
```

- Call:

```
DECLARE
   v_salary NUMBER := 40000;
BEGIN
   increase_salary(v_salary);
   DBMS_OUTPUT.PUT_LINE('New Salary = ' || v_salary);
END;
/
```

- **Output**: New Salary = 45000
- **Example 5**: Procedure to Display Employee Details

```
CREATE OR REPLACE PROCEDURE display_employee
(p_emp_id IN NUMBER) IS
   v_name employee.emp_name%TYPE;
   v_salary employee.salary%TYPE;
BEGIN
   SELECT emp_name, salary INTO v_name, v_salary FROM employee
   WHERE emp_id = p_emp_id;
   DBMS_OUTPUT.PUT_LINE('Employee ID : ' || p_emp_id);
   DBMS_OUTPUT.PUT_LINE('Name       : ' || v_name);
   DBMS_OUTPUT.PUT_LINE('Salary     : ' || v_salary);
EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Employee not found.');
END;
/
```

- **Execute**: EXEC display_employee(101);

Parameterized procedure

- **In Mode**

```
Create or replace procedure add_dept
(p_did IN department.dept_id%type, p_dname IN
department.dept_name%type, p_mid IN department.mgr_id%type, p_lid IN
department.location_id%type)
IS
Begin
        Insert into department values (p_did, p_dname, p_mid, p_lid);
        dbms_output.put_line('TEST added');
End;
```

Procedure created.

```
Execute add_dept(200, 'Civil', 104, 10);
```

Department added.

- **OUT mode**

```
Create or replace procedure add_dept
(p_did IN department.dept_id%type, p_dname OUT
department.dept_name%type)
IS
Begin
        Select dept_name into p_dname from department where dept_id = pdid;
End;
```

Procedure created.

```
Declare
    Dname varchar(20);
Begin
    Add_dept(500, dname);
    Dbms_output.put_line(dmame);
End;
/
```

- **IN OUT mode**

```
create or replace procedure phone_no(p_phone_no IN OUT varchar2) IS
begin
    p-phone_no := '(' || SUBSTR(p_phone_no, 1, 3) || ')' ||
SUBSTR(p_phone_no, 1, 3) || '-' || SUBSTR(p_phone_no, 7) ;
end;
/
```

Procedure created.

```
Declare
    V_p_no varchar2(15);
Begin
    V_p_no := '1234567890';
    Phone_no(v_p_no);
    Dbms_output.put_line(v_p_no);
End;
/
```

Output: (123)456-7890

- **Another Example**:

```
Create or replace procedure getTopperStudents AS
TopperName students.stud_name%type;
begin
select stud_name into TopperName from students where dbms=(select max(dbms)
from students)
dbms_output.put_line(TopperName);
end;
/
Begin
getTopperStudents;
end;
/
```

Assignment

1. Create the procedure: increment_salary(emp_id, percentage)
   The procedure should:
   a. Check whether the employee exists.
   b. Calculate the increment.
   c. Update the salary.
   d. Display old salary.
   e. Display increment.
   f. Display new salary.
   g. Handle employee-not-found condition.